Introduction

Recruiting and retaining drivers is seen by industry watchers as a tough battle for Ola. Churn among drivers is high and it’s very easy for drivers to stop working for the service on the fly or jump to Uber depending on the rates.

As the companies get bigger, the high churn could become a bigger problem. To find new drivers, Ola is casting a wide net, including people who don’t have cars for jobs. But this acquisition is really costly. Losing drivers frequently impacts the morale of the organization and acquiring new drivers is more expensive than retaining existing ones.

You are working as a data scientist with the Analytics Department of Ola, focused on driver team attrition. You are provided with the monthly information for a segment of drivers for 2019 and 2020 and tasked to predict whether a driver will be leaving the company or not based on their attributes like

Problem Statement

customer

Airport

ola

uber/meru

fasttork

① Either no cabs are available
② Drivers cancelled the ride.
→ Some problem

OLA → cab sharing
OLA → ② Loss of business

① customer will not be happy with the OLA service provider

+ → Drivers not happy
+ → OLA doesn't have enough partners

Solution

① OLA should analyze the pool of current drivers and check which all of them are about to churn

Retain drivers

Imputation Techniques

1. mean / median / mode
2. Iterative Imputer
3. KNN Imputer
4. Imputation using Regression
5. Imputation using NNs

=

Data

per month → per Drivers level

1. Income
2. Rating
3. Date of Joining
- ⋮
- n

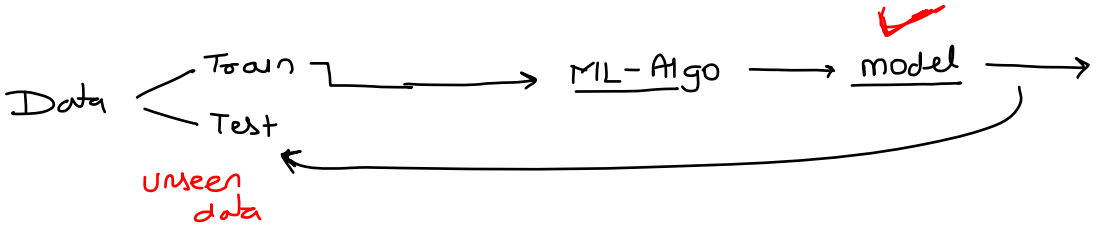
→ Pattern about a drivers leaving the company / ola

y → categorical → classification algo →
1. Random forest
2. Boosting
3. Decision tree.

↓

model → OLA → predict which all drivers are about to churn

=



Train (Sensitivity) → 0.85
Test (Sensitivity) → 0.50
} X

Cross-validation

metric
Train (Sensitivity) → 0.85
Test (Sensitivity) → 0.80
} → good model

"Occam's Razor"
↓
model → simplest and is able to perform sufficiently good

ML-problem → classification
Regression

Baseline model (Simplest model) → Increase complexity

"Logistic Regression"

final model.

⇒ End objective.

→ Business explaination ⟶ Independent features ∩ dependet feature

→ Deployment

ML-

Logistic Regression
Decission Regression

PCA

Random forest
Boosting
→ NN

manger

$y = \beta_0 + \beta_1(x_1) + \beta_2(x_2)$

kaggle ⟶ objective ⟶ Best performace score.

Industry ⟶ objective → Explain ⟶ [———]

deploy ⟶ [→ → → →]

x_1 x_2 x_3 → (y)

→ Best out of the data

Class - Imbalance problem ?

Real classification Data

X	y
	0 → 92 %
	1 → 8 %

Train

class-Imbalance

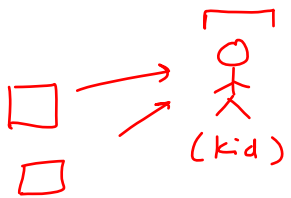
→ [ML Algo] → Model → works very well on class-0

→ fails on class-1

(the algo would be able to learn the pattern associated with class-0, very well and for class-1, the pattern would not be learnt properly)

Credit card data

y (x)
- 1 → defaulted → 8 %
- 0 → not default → 92 %



class-Balancing techniques

1. Random oversampling
2. Random undersampling
3. SMOTE ⟶
4. class-weight
5. Ada-Syn

Class-weight

→ cost function for Logistic Regression

$$\mathcal{L}(\beta) = -\sum_{i=1}^{m} [\, y \log(y_\beta(x)) + (1-y) \log(1-y_\beta(x)) \,]$$

$$\text{cost}(y_\beta(x), y_i) = \begin{cases} -\log(y_\beta(x)) & \text{if } y_i = 1 \\ -\log(1-y_\beta(x)) & \text{if } y_i = 0 \end{cases}$$

y_i = actual label

y < 0 : 80%, 1 : 20%

class-weight = "balanced" → {0 : 0.2, 1 : 0.8}

class-weight

$$\text{cost}(y_\beta(x), y_i) = \begin{cases} -0.8 \log(y_\beta(x)) & \text{if } y_i = 1 \\ -0.2 \log(1-y_\beta(x)) & \text{if } y_i = 0 \end{cases}$$

forcing the algo to focus more on class-1 than on class-0

If the algo misclassifies a +ve class (1), then there would be more penalty as compared with -ve (0)

Variance (data) $\propto$ Information content

Baseline model
(No hyperparameter tuning)

↓

77% ⟶ 88% ⟶ 90%
+12%

1. missing value treatment
2. feature engineering
3. Data preprocessing
4. choice of model/algo
5. Hyperparameters
6. C-V

Ex Titanic ⟶ 0.78% Sensitivity ⟶ 100%

↙ ↓

80% ⟶ 92%
+4%

100%